CSC 335 Group Project

Sophia Harms, Jesse Catterton, Tyler Ohme

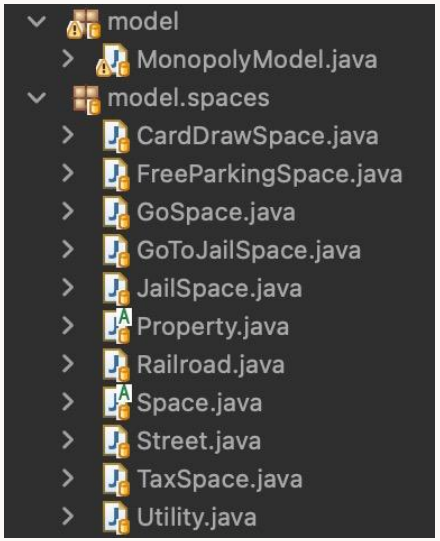
Monopoly Team 2

Final Video Presentation Link: https://youtu.be/HJOKjgapnx0

Longer Video with Gameplay Presentation Link: https://youtu.be/lZ5Va2hQ5GU

Package Design Choices





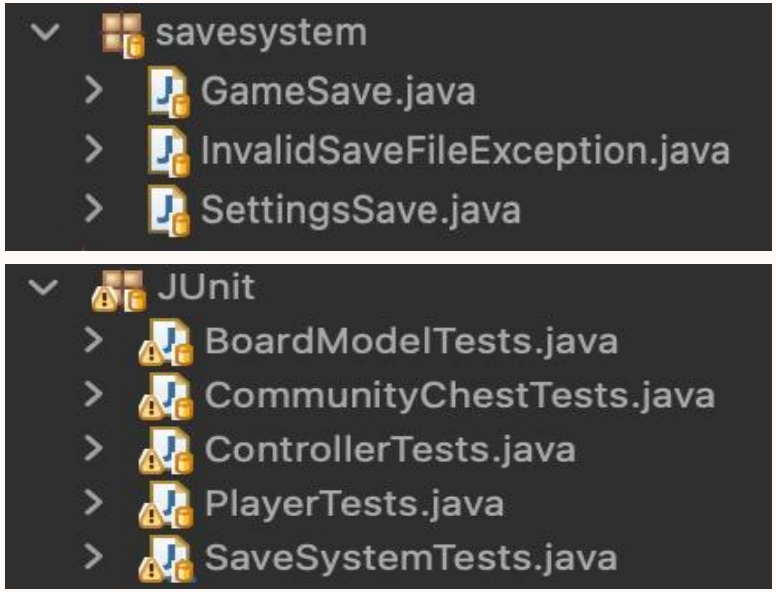
Monopoly Package:

- Classes for view and controller related classes.
- Each class specializes in one aspect of the gameplay

Model Package:

- Classes for gameplay logic of each tile type

Package Design Choices



Save System Package:

- Classes for saving the game
- Specialized functionality of the game that does not affect the game logic

JUnit Package:

- Individual JUnit classes to test specific classes in previous packages
- Targeted testing classes

Starting the Game

```
public void startGame() {
    players = controller.getPlayers();

    if (!controller.isLoadedFromSave()) {
        controller.setPlayerBalance(players); // sets starting balance
        controller.setCurPlayerIndex(0); // sets player index to first slot
    }

    addPlayerProfiles(); // add player bar on side
    ((GridPane) playerProfiles.get(controller.getCurPlayerIndex())).setStyle("-fx-background-color: #949494; -fx-border-color: black; -fx-border-width: 2px;");
    updatePlayerBalances();
    for (int i = 0; i < humanPlayerCount + computerPlayerCount; i++) { // loops to move each player to go or to starting location from save
        if (players[i].getLocation() == 10 && players[i].checkJailed()) { // places player in jail if they were in jail on loaded game
            CornerTile jailTile = (CornerTile) tiles[10];
            tiles[30].moveFrom(players[i]);
            jailTile.sendToJail(players[i]);
        } else tiles[players[i].getLocation()].moveTo(players[i]);
    }

    cardDeck = new CommunityChest(theme);
    addLogos();
```

Design Process:

- Check if a game is being loaded from a save
- Build the GUI board
- Add the themed Community Chest

Player Movement

```
public void movePlayer() {
    Player currentPlayer = players[controller.getCurPlayerIndex()];
    save.setDisable(true); // disable button to prevent abusing save/load to roll multiple times

    boolean disableEndTurn = false;
    if (currentPlayer.checkJailed()) { // completes a jail turn if player is in jail
        completeJailedTurn(currentPlayer);
        rollDice.setDisable(true);
        if (controller.getTrading()) proposeTrade.setDisable(false); // only enables trading button if trading is allowed
        build.setDisable(false);
        sell.setDisable(false);
        endTurn.setDisable(disableEndTurn);
    } else { // completes normal turn
        int oldLocation = currentPlayer.getLocation();
        int diceRoll[] = currentPlayer.diceRoll(); // display dice
        AudioClip rolling = new AudioClip(getClass().getResource("/diceRoll.wav").toExternalForm());
        rolling.play();
        dieView1.setImage(dieFaces[diceRoll[0] - 1]);
        dieView2.setImage(dieFaces[diceRoll[1] - 1]);
        animatePlayer(currentPlayer, diceRoll[0] + diceRoll[1], oldLocation);
    }
}
```

Design Process:

- Check if Player is in jail
- If they are complete a specialized jail turn and dont allow movement
- If they are not in jail, roll two dice and animate player movement

Animation

```java
public void animatePlayer(Player currPlayer, int roll, int oldLocation) {
    rollDice.setDisable(true);
    Timeline timeline = new Timeline();
    timeline.setCycleCount(1);
    int currLocation = oldLocation;

    int seconds = 250;
    for (int i = currLocation; i < (currLocation + roll); i++) { // loops to step for each roll
        int step = i;
        KeyFrame keyframe = new KeyFrame(Duration.millis(seconds), (event) -> {
            if (step > 39) tiles[step - 40].moveFrom(currPlayer);
            else tiles[step].moveFrom(currPlayer);

            if (step + 1 > 39) tiles[step + 1 - 40].moveTo(currPlayer);
            else tiles[step + 1].moveTo(currPlayer);
        });
        timeline.getKeyFrames().add(keyframe);
        seconds += 250;
    }
    timeline.setOnFinished((event) -> Platform.runLater(() -> { // credit to ta jacob for this line

        int newLocation = currPlayer.getLocation();

        if (newLocation < oldLocation) { // checks if player passes go
            currPlayer.adjustBalance(controller.getGoAmount());
            updatePlayerBalances();
            AudioClip passGo = new AudioClip(getClass().getResource("/go.wav").toExternalForm());
            passGo.play();
        }

        boolean disableEndTurn = interpretSpace(currPlayer);

        if (controller.getTrading()) proposeTrade.setDisable(false); // only enables trading button if trading is allowed
        build.setDisable(false);
        sell.setDisable(false);
        endTurn.setDisable(disableEndTurn);

        if (currPlayer instanceof ComputerPlayer) {
            endTurn();
        }
    }));
    timeline.play();
}
```

Design Process:

- Timeline Object to hold KeyFrame Objects
- KeyFrame added for every tile the player moves
- Simulates look of picking up and setting down your piece repeatedly as you skip to where you landed

Player's Landing Space

```
public boolean interpretSpace(Player currentPlayer) {
    Tile currTile = tiles[currentPlayer.getLocation()];
    Space space = currTile.getSpace();

    if (space instanceof Property) { // handles landing on a property space
        return handlePropertySpace(space, currentPlayer);
    } else if (space instanceof CardDrawSpace) {  // handles landing on a community chest space
        drawCard(currentPlayer);
    } else if (space instanceof FreeParkingSpace) { // collects free parking if its enabled
        if (controller.getFreeParking()) {
            FreeParkingSpace freeParking = (FreeParkingSpace) space;
            int reward = freeParking.collectRewards();
            currentPlayer.adjustBalance(reward);
        }
        AudioClip carHonk = new AudioClip(getClass().getResource("/freeParking.wav").toExternalForm());
        carHonk.play();
    } else if (space instanceof GoToJailSpace) { // sends player to jail
        CornerTile jailTile = (CornerTile) tiles[10];
        tiles[30].moveFrom(currentPlayer);
        jailTile.sendToJail(currentPlayer);
        currentPlayer.setLocation(10);
        currentPlayer.setJailed(true);
        currentPlayer.setJailRollCount(0);
        AudioClip jailDoor = new AudioClip(getClass().getResource("/jail.wav").toExternalForm());
        jailDoor.play();
    } else if (space instanceof TaxSpace) { // pays taxes
        int taxAmount = ((TaxSpace) space).getTaxAmount();
        currentPlayer.adjustBalance(-taxAmount);
        if (controller.getFreeParking()) { // puts tax money in free parking if its enabled
            FreeParkingSpace freeParking = (FreeParkingSpace) tiles[20].getSpace();
            freeParking.addRewards(taxAmount);
        }
    }
    updatePlayerBalances();
    return false;
}
```

Design Process:

- Check if tile is an instance of a:
  - Property
  - Community Chest
  - Free Parking
  - Go to Jail
  - Tax Space
- Specialzed methods called for specific tiles
- Player balances on the GUI are updated at the end

Landing on Property

```java
public boolean handlePropertySpace(Space space, Player currentPlayer) {
    Player owner = ((Property) space).getOwner();
    if (space instanceof Railroad) {
        AudioClip railroad = new AudioClip(getClass().getResource("/railroad.wav").toExternalForm());
        railroad.play();
    }
    if (owner == currentPlayer) { // do nothing if player owns the property
        return false;
    } else if (owner == null) { // allow player to purchase property
        if (currentPlayer instanceof ComputerPlayer && ((ComputerPlayer) currentPlayer).makePropertyDecision(((Property) space).getBuyValue())) {
            currentPlayer.buyProperty((Property) space); // computer buys property if it can afford it
        }
        buyProperty.setDisable(false);
        if (!controller.getSkipPurchases() && currentPlayer.getBalance() >= ((Property) space).getBuyValue()) return true; // returns true if endTurn should be disabled
    } else { // make player pay rent to ownder
        int diceNum = currentPlayer.getLastRoll();
        int rentPrice = ((Property) space).getRent(diceNum);

        // Have current player pay rent to the owner as long as the owner isn't in jail
        if (!owner.checkJailed()) {
            currentPlayer.adjustBalance(-rentPrice);
            owner.adjustBalance(rentPrice);
        }
    }
    updatePlayerBalances();
    return false;
}
```

Design Process:

- If no owner -> allow choice to buy property (Computer Players must buy property)
- If owner -> pay rent to owner if they are not in jail
- Update player's balances on the GUI

End Turn

```
public void endTurn() {
    save.setDisable(false); // disable button to prevent abusing save/load to roll multiple times

    if (players[controller.getCurPlayerIndex()].checkBankrupt() && canEscapeBankrupcy(players[controller.getCurPlayerIndex()])) { // checks if player is bankrupt and can get out
        if (players[controller.getCurPlayerIndex()] instanceof ComputerPlayer) { // sells properties automatically for computer
            while (!players[controller.getCurPlayerIndex()].getProperties().isEmpty()) { // loops to sell as many properties as necessary to get out of bankrupcy
                    Property property = players[controller.getCurPlayerIndex()].getProperties().get(0);
                    sellProperty(property, sellList, false);
                    if (!players[controller.getCurPlayerIndex()].checkBankrupt()) break; // breaks once computer gets out of bankrupcy
            }
        } else { // shows message for player to manually sell properties
            Alert msg = new Alert(Alert.AlertType.ERROR);
             msg.setHeaderText("Bankrupt Player");
             msg.setContentText("You may not end your turn with a negative balance. Sell properties to continue.");
             msg.showAndWait();
             return;
        }
    }

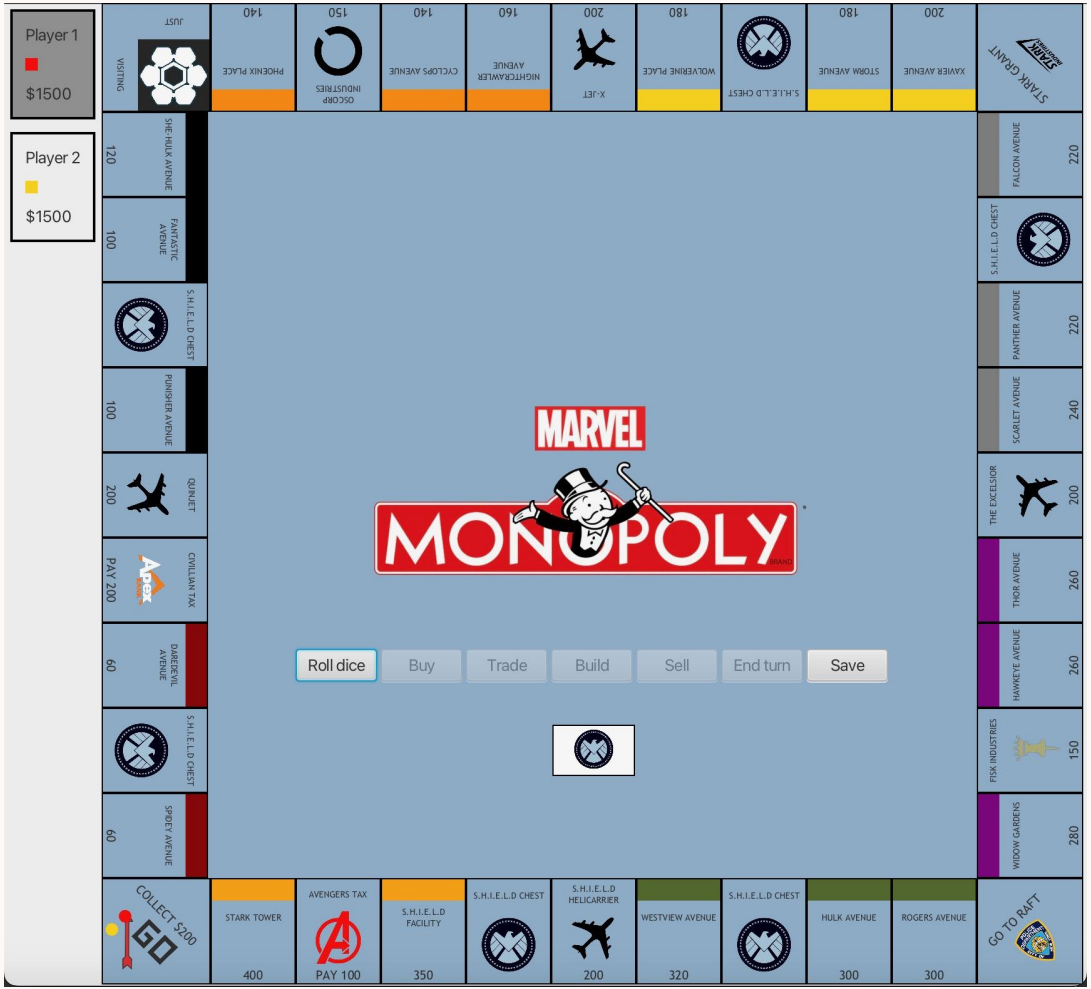
    Player winner = controller.checkGameOver();
    if (winner != null) { // checks if winner is present
        Alert msg = new Alert(Alert.AlertType.INFORMATION);
        msg.setTitle("Game Over");
        msg.setHeaderText("Game Over");
        msg.setContentText("Player " + (winner.getPlayerId() + 1) + " wins!");
        msg.showAndWait();
        System.exit(0); // ends program once they exit out of win message
    }

    ((GridPane) playerProfiles.get(controller.getCurPlayerIndex())).setStyle("-fx-background-color: #00000000; -fx-border-color: black; -fx-border-width: 2px;");
    if (!players[controller.getCurPlayerIndex()].getRollWasDoubles()) { // doesn't move to next player if roll was doubles
        controller.nextPlayerIndex();
        if (controller.getCurPlayerIndex() >= players.length) controller.setCurPlayerIndex(0); // loops back to first player
        while (players[controller.getCurPlayerIndex()].checkBankrupt()) {
            controller.nextPlayerIndex();
            if (controller.getCurPlayerIndex() >= players.length) controller.setCurPlayerIndex(0); // loops back to first player
        }
    }
    ((GridPane) playerProfiles.get(controller.getCurPlayerIndex())).setStyle("-fx-background-color: #949494; -fx-border-color: black; -fx-border-width: 2px;");
    rollDice.setDisable(false);
    build.setDisable(true);
    proposeTrade.setDisable(true);
    endTurn.setDisable(true);
    buyProperty.setDisable(true);
    sell.setDisable(true);
    updatePlayerBalances();
    if (players[controller.getCurPlayerIndex()] instanceof ComputerPlayer)  {
        rollDice.setDisable(true);
        computerPlayerTurn();
    }
}
```

Design Process:

- Prevent save abuse
- Check bankruptcy (Force player to sell to get out of it. Computers automatically sell properties)
- Check game over logic
- If over -> display winner and exit
- If not -> check if doubles rolled to have another turn. Next player's turn.

Marvel Theme



DC Comics Theme



Demo Time!